计算机组成与结构
—计算机中的数据表示2

李瑞　　教授
蒋志平　副教授
计算机科学与技术学院

温故一 祭祖传统娱乐节目

- 10进制转X进制整数部分怎么转?
- 10进制转X进制小数部分怎么转?
- 2进制向8/16进制转换的规则是?
- 原码怎么转反码?
- 反码怎么转补码?
- 补码怎么转移码?
- 0.25的补码是多少? （用4位定长纯小数表示）
- -0.25的补码是多少? （用4位定长纯小数表示）
- 0.5的补码是多少? （用4位定长纯小数表示）
- -0.5的补码是多少? （用4位定长纯小数表示）

本章学习要点

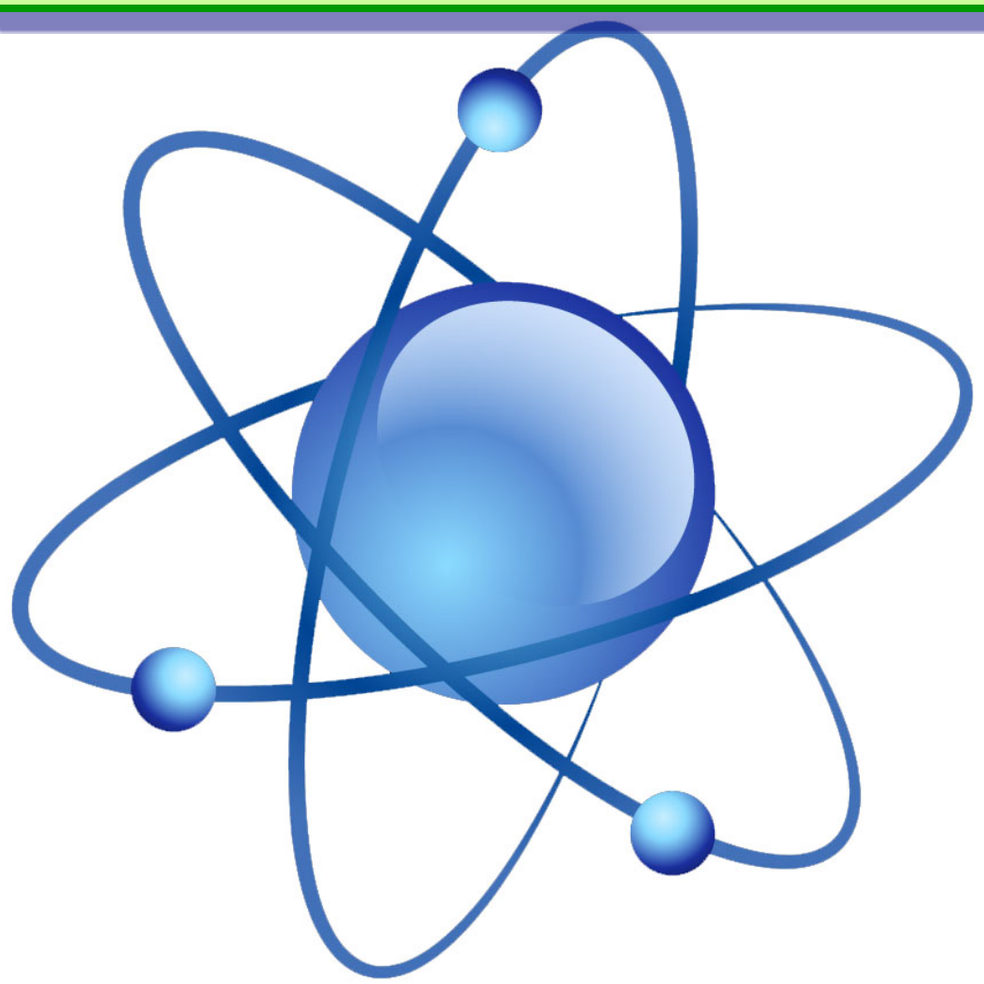
- 数字的表示
  - 进制编码基础
  - 整数及小数进制转换（给人看）
  - 整数及小数在计算机里的表示（给机器看）
- 有符号号的原、反、补、移码
- 浮点表示
  - IEEE754
- BCD

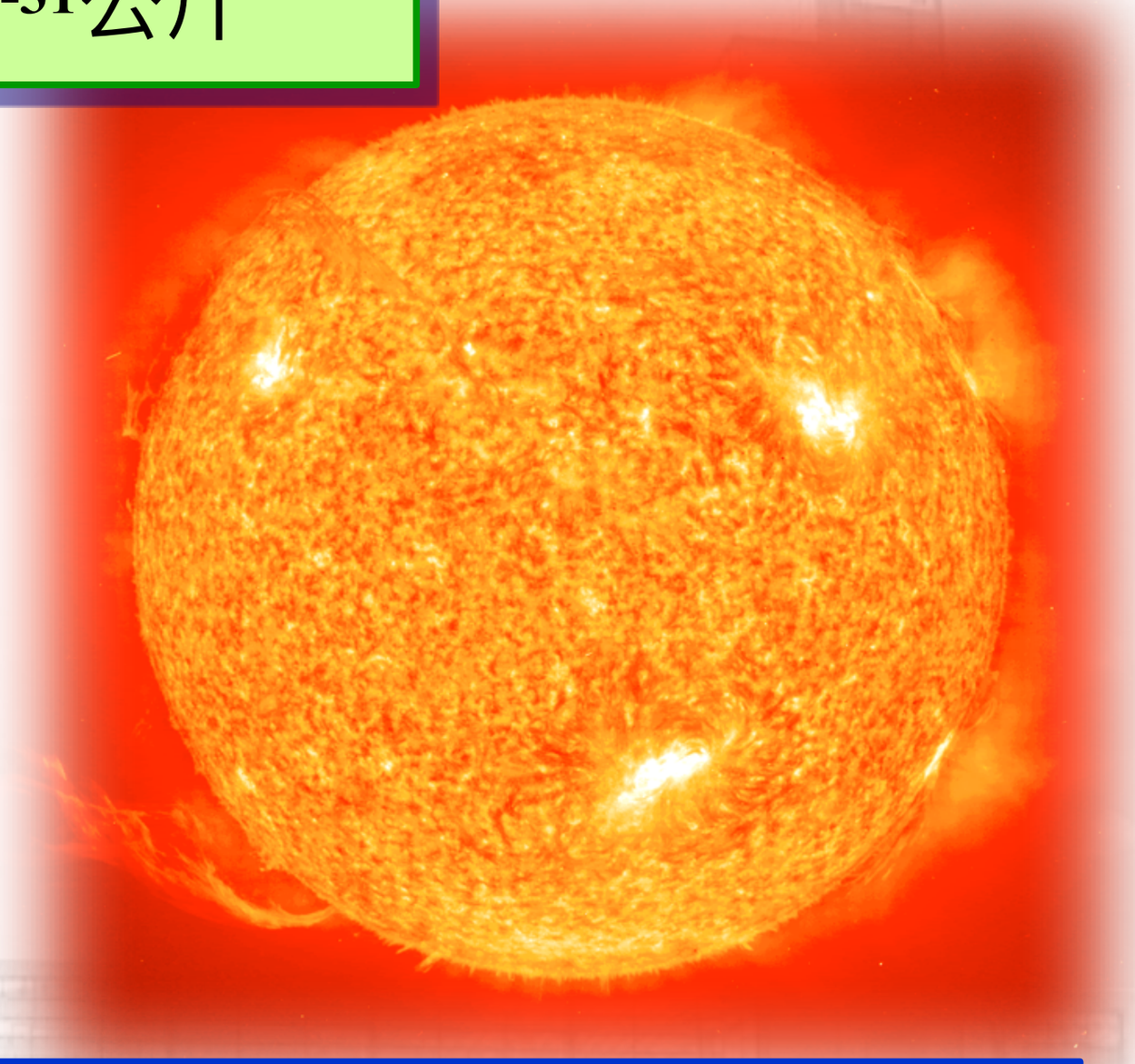
- 非数字的表示
  - 字符表示
    - ASCII
    - 汉字
    - Unicode
- 校验码
  - 奇偶校验
  - CRC

浮点表示基础

如何表示典型的实数?

电子的质量＝9.109 38×10^{-31}公斤



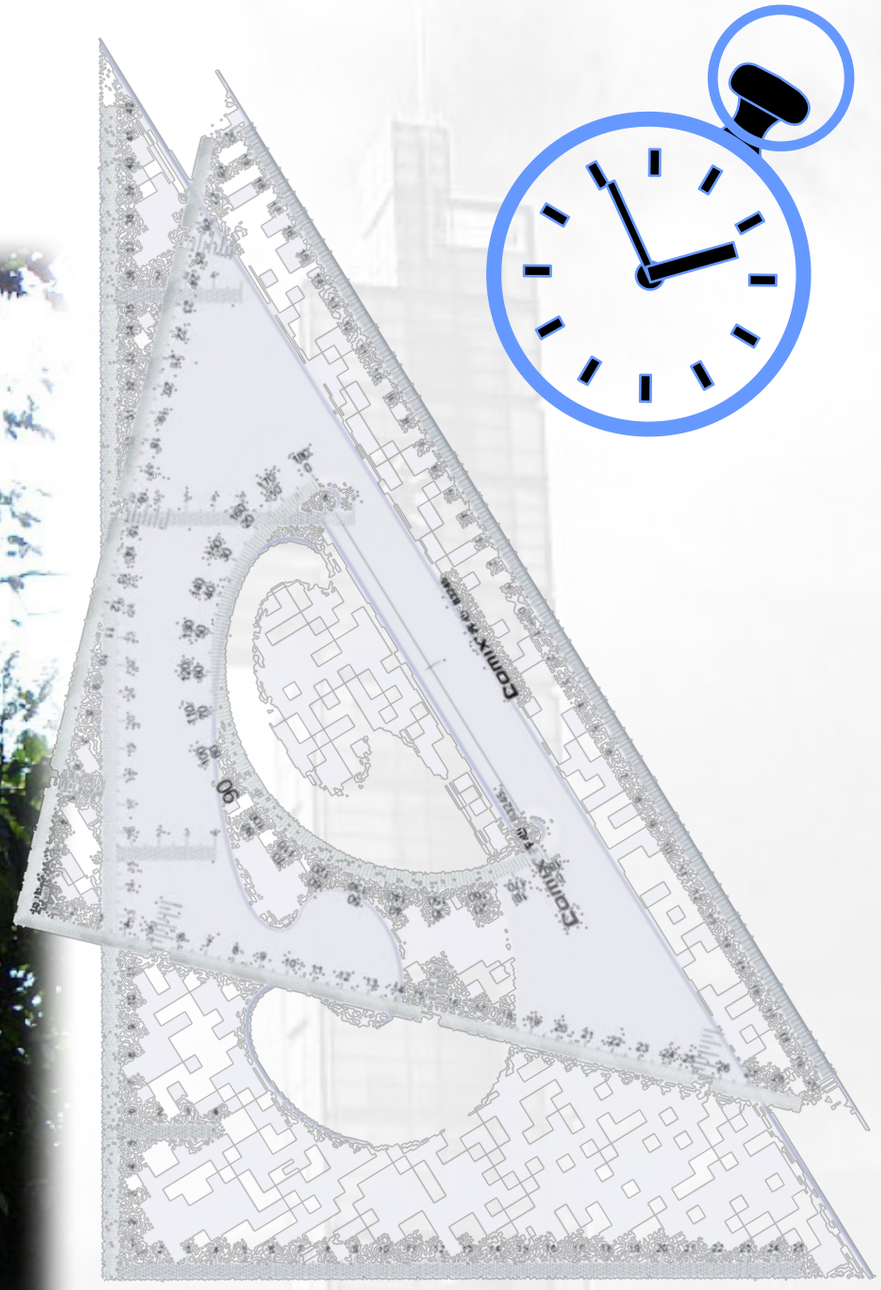


太阳的质量＝1.988 92×10^{30}公斤

如何表示典型的实数?

1世纪＝3.155 76×10^{9}秒





1纳秒＝1.0×10^{-9}秒

计算机如何表示实数?

- 计算机里表达实数的方法?
  - 有限位数表示(Fixed Length)
    - 分数法—精确表达有理数（3/1, 22 / 7, 355 / 113，...)
    - 无理数靠邻近有理数表示（$\pi \approx \frac{355}{113}$,...)
  - 小数法
    - 定点小数（定长整数位+定长小数位）<—常规小数法
    - 浮点小数（定长阶数位+定长小数位）<—科学计数法
- 可变位数表示 (Variable Length)

计算机如何表示实数?

数域	表示方式	bit位用途		
整数	无符号整数(Unsigned Integer)	整数		
	有符号整数(Signed Integer)	符号	整数	
	可变长表示法(Variable Length)	符号	位数（可变长）	数字（可变长）
实数	无符号有理数(Unsigned Rational)	分子		分母
	有符号有理数(Signed Rational)	符号	分子	分母
	无符号定点小数(Unsigned Fixed-Point)	整数		小数
	有符号定点小数(Signed Fixed-Point)	符号	整数	小数
	浮点小数(Floating Point)	**符号**	**阶数**	**尾数**

浮点表示

小数F的浮点表示（科学计数法表示）如下：

$$F = M \times R^E$$

- M，mantissa尾数；R, radix基数；E， exponent 阶数（幂级）
- M和为有符号小数；E为有符号整数
- R，当然是2
- 数学上，M、R、E可以是任意组合，比如，

$$\begin{aligned}(20.3)_{10} &= (10100.010011..)_B \\ &= 10100.010011 \times 2^0 \\ &= 101.00010011 \times 2^{10} \\ &= 1.0100010011 \times 2^{100} \\ &= 0.010100010011 \times 2^{110} \\ &\cdots\end{aligned}$$

浮点表示

在计算机表达浮点数，有以下问题：

- 尾数M怎么表示?
  - 总不能把尾数M再用浮点表示吧~?
- bit位怎么规定?
  - 几位bit尾数? 几位bit阶数?
    - 精度 v.s 范围
  - 阶数与尾数的位置?
    - 阶符-阶码-数符-尾数?
    - 数符-阶符-阶码-尾数?
    - ....
- M、E用什么表示方式?
  - 原? 反? 补? 移?
    - 这真不一定...

例如:

$$
\begin{aligned}
(20.3)_{10} &= (10100.010011..)_B \\
&= 10100.010011 \times 2^0 \\
&= 101.00010011 \times 2^{10} \\
&= 1.0100010011 \times 2^{100} \\
&= 0.010100010011 \times 2^{110} \\
&\cdots
\end{aligned}
$$

浮点表示—尾数与阶数的表示

针对第一个问题：

- 尾数M怎么表示?
  - 总不能把尾数M再用浮点表示吧~?

例如:

$$
\begin{aligned}
(20.3)_{10} &= (10100.010011..)_B \\
&= 10100.010011 \times 2^0 \\
&= 101.00010011 \times 2^{10} \\
&= 1.0100010011 \times 2^{100} \\
&= 0.010100010011 \times 2^{110} \\
&\cdots
\end{aligned}
$$

解决方案：**调整E，将尾数M变成纯小数**

纯小数尾数:

$$
\begin{aligned}
(20.3)_{10} &= (10100.010011..)_B \\
&= 10100.010011 \times 2^0 \\
&= 0.10100010011 \times 2^{101} \\
&= 0.0010100010011 \times 2^{111} \\
&\cdots
\end{aligned}
$$

浮点表示—尾数与阶数的表示

针对第二个问题：

- bit位怎么规定?
  - 几位bit尾数? 几位bit阶数?
    - 精度 v.s 范围

- 有限位尾数M，有限位阶数E
  - 但不同的表示方式有不同的精度损失；

解决方案：规格化（Normalization）

无限精度尾数

$$
\begin{aligned}
(20.3)_{10} &= (10100.010011..)_B \\
&= 10100.010011 \times 2^0 \\
&= 0.10100010011 \times 2^{101} \\
&= 0.0010100010011 \times 2^{111} \\
&\cdots
\end{aligned}
$$

有限位尾数 + 有限位阶数

$$
\begin{aligned}
(20.3)_{10} &= (10100.0100110011..)_B \\
&= 0.10100010011 \times 2^{0101} \\
&= (20.296875)_D \\
&= 0.01010001001 \times 2^{0110} \\
&= (20.28125)_D \\
&= 0.00101000100 \times 2^{0111} \\
&= (20.25)_D \\
&= 0.00000101000 \times 2^{1010} \\
&= (20.0)_D \\
&\cdots
\end{aligned}
$$

浮点表示—尾数与阶数的表示

针对第二个问题：

- bit位怎么规定?
  - 阶数与尾数的位置?
    - 阶符-阶码-数符-尾数?
    - 数符-阶符-阶码-尾数?
    - ...

解决方案：主要有两种

阶符—阶码—数符—尾数，(不知名格式)

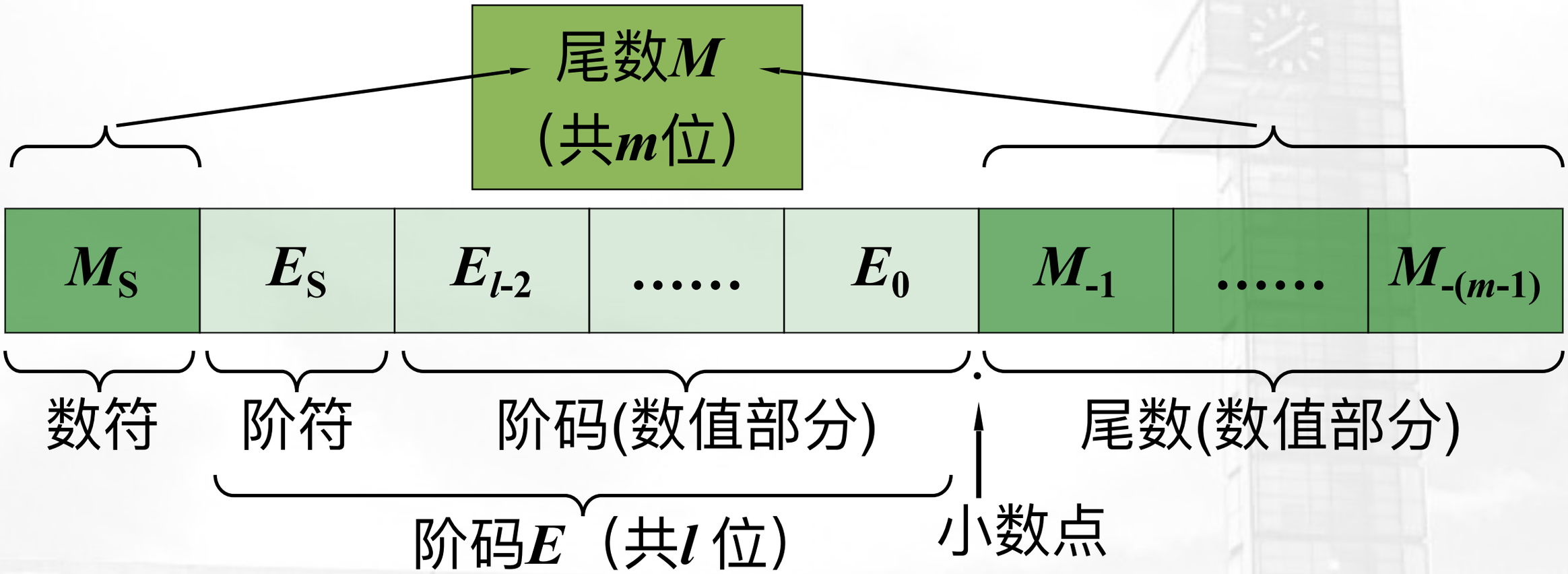
数符—阶符—尾数，(IEEE754格式)

有限位尾数 + 有限位阶数

$$
\begin{aligned}
(20.3)_{10} &= (10100.0100110011..)_B \\
&= 0.10100010011 \times 2^{0101} \\
&= 0.01010001001 \times 2^{0110} \\
&= 0.00101000100 \times 2^{0111} \\
&= 0.00000101000 \times 2^{1010} \\
&\cdots
\end{aligned}
$$

浮点表示—尾数与阶数的表示

- **IEEE754风格浮点数**



浮点表示—尾数与阶数的表示

针对第二个问题：

- M、E用什么表示方式?
  - 原? 反? 补? 移?
    - 这真不一定...

就两种情况：

- 题目要求怎么来，就怎么来😢
  - 注意审题哦~
- 严格确定的IEEE754格式

浮点表示—规格化
(Normalization)

浮点表示—规格化

当采用有限尾数+有限阶数表示时，不同

表示方法的精度损失不同：

$$
\begin{aligned}
(20.3)_{10} &= (10100.0100110011..)_B \\
&= 0.10100010011 \times 2^{0101} \\
&= (20.296875)_D \\
&= 0.01010001001 \times 2^{0110} \\
&= (20.28125)_D \\
&= 0.00101000100 \times 2^{0111} \\
&= (20.25)_D \\
&= 0.00000101000 \times 2^{1010} \\
&= (20.0)_D \\
&\cdots
\end{aligned}
$$

怎样的M与E能最好地保持精度?

浮点表示—规格化

当采用有限尾数+有限阶数表示时，不同

表示方法的精度损失不同：

$$(20.3)_{10} = (10100.0100110011..)_B$$
$$= 0.10100010011 \times 2^{0101}$$
$$= (20.296875)_D$$
$$= 0.01010001001 \times 2^{0110}$$
$$= (20.28125)_D$$
$$= 0.00101000100 \times 2^{0111}$$
$$= (20.25)_D$$
$$= 0.00000101000 \times 2^{1010}$$
$$= (20.0)_D$$
$$\cdots$$

怎样的M与E能最好地保持精度?

最优规则：

- 尾数头部没有不必要的0 —— 规格化

=> 尾数M满足，$0.5 \le \|M\| \le 1$

$$0.10100010011 \times 2^{0101} = (20.296875)_D$$

浮点表示—规格化

浮点数规格化，Floating Point Normalization,就是调整M和E，让M头部没有不必要的0，最优地保持精度...

浮点表示—规格化

浮点数规格化，Floating Point Normalization,就是调整M和E，让M头部没有不必要的0，最优地保持精度...

嘛时候，M头部有0，而且还是最优精度保持~?

浮点表示—规格化

当尾数M用补码表示时，M应符合，

$0.5 \le \|M\| \le 1$，得：

- 当$M > 0$且$0.5 \le \|M\| < 1$时，
  - 当$[M]_{原} = 0.1XXX$时，$[M]_{补} = 0.1XXX$

浮点表示—规格化

当尾数M用补码表示时，M应符合，

$0.5 \le \|M\| \le 1$，得：

- 当$M > 0$且$0.5 \le \|M\| < 1$时，
  - 当$[M]_{原} = 0.1XXX$时，$[M]_{补} = 0.1XXX$
- 当$M < 0$且$0.5 < \|M\| \le 1$时，
  - 当$[M]_{原} = 1.1XXX$时，$[M]_{补} = 1.0XXX$

0.5是个例外

$$[-0.1000]_{补} = 1.1000$$
$$[-0.1001]_{补} = 1.0111$$
$$[-0.1010]_{补} = 1.0110$$
$$[-0.1011]_{补} = 1.0101$$
$$[-0.1100]_{补} = 1.0100$$
$$[-0.1101]_{补} = 1.0011$$
$$[-0.1110]_{补} = 1.0010$$
$$[-0.1111]_{补} = 1.0001$$
$$[-1.0000]_{补} = 1.0000$$

浮点表示—规格化

所以，当尾数M用补码表示时，规格化的M应符合，

- 当$M > 0$时，M符合0.1XXXXX形式
- 当$M < 0$时，M符合1.0XXXXX形式
- M=-0.5不是规格化
- M=-1.0是规格化
  - 当M=-0.5时，把M放大到-1.0

0.5是个例外

$$[-0.1000]_{补} = 1.1000$$
$$[-0.1001]_{补} = 1.0111$$
$$[-0.1010]_{补} = 1.0110$$
$$[-0.1011]_{补} = 1.0101$$
$$[-0.1100]_{补} = 1.0100$$
$$[-0.1101]_{补} = 1.0011$$
$$[-0.1110]_{补} = 1.0010$$
$$[-0.1111]_{补} = 1.0001$$
$$[-1.0000]_{补} = 1.0000$$

浮点表示—规格化

如何得到规格化浮点表示数?

1. 得到基本浮点表示（注意M、E的表示法）
2. 若M使用原码表示：M左/右移n位使符合$0.5 \le \|M\| < 1$，注意E是否溢出
3. 若M使用补码表示：
   - 当M>0时，参考原码方案；
   - 若M用补码表示且M<0时，建议解决方案如下：
     - 把M按原码调整至$0.5 < \|M\| \le 1$范围
     - M转为补码

浮点表示—规格化

【例1】将十进制数173507转换为下述IBM370的短浮点数格式（32位）

31	30 … 24	23 … 0
S	指数(阶码E)	尾数M
1位	7位	24位

尾数M：6位十六进制原码表示；尾数的基B＝16，阶码加1（或减1）相当于尾数右移（或左移）4位。

阶码E：7位移码表示

数符S

浮点表示—规格化

【例1】将十进制数173507转换为下述IBM370的短浮点数格式（32位）

31	30 ～ 24	23 ～ 0
S	指数(阶码E)	尾数M
1位	7位	24位

【解】$(173507)_{10}=(2A5C3)_{16}=(0.2A5C30)_{16}\times16^5$

所以数符$S=0$，阶码$E=(64+5)_{10}=(1000101)_2$

因此$(173507)_2$的IBM370的短浮点数格式为：

31	30 ～ 24	23 ～ 0
S	1000101	0010 1010 0101 1100 0011 0000
1位	7位	24位

浮点表示—规格化

【例2】将十进制数173507转换为下述典型的32位浮点数格式。

31	30 … 23	22 … 0
S	指数(阶码E)	尾数M
1位	8位	23位

尾数M：24位二进制原码表示；规格化尾数的第一位总是1，无需保存(用23位尾数表示24位数据)；

尾数的基数B＝2。

阶码E：8位移码表示，偏置常数为(10000000)2。

数符S

浮点表示—规格化

【例2】将十进制数173507转换为下述典型的32位浮点数格式。

31	30 … 23	22 … 0
S	指数(阶码E)	尾数M
1位	8位	23位

【解】$(173507)_{10}=(2A5C3)_{16}$

$=(0010\ 1010\ 0101\ 1100\ 0011)_2$

$=(0.1010\ 1001\ 0111\ 0000\ 1100\ 0000)_2\times 2^{17}$

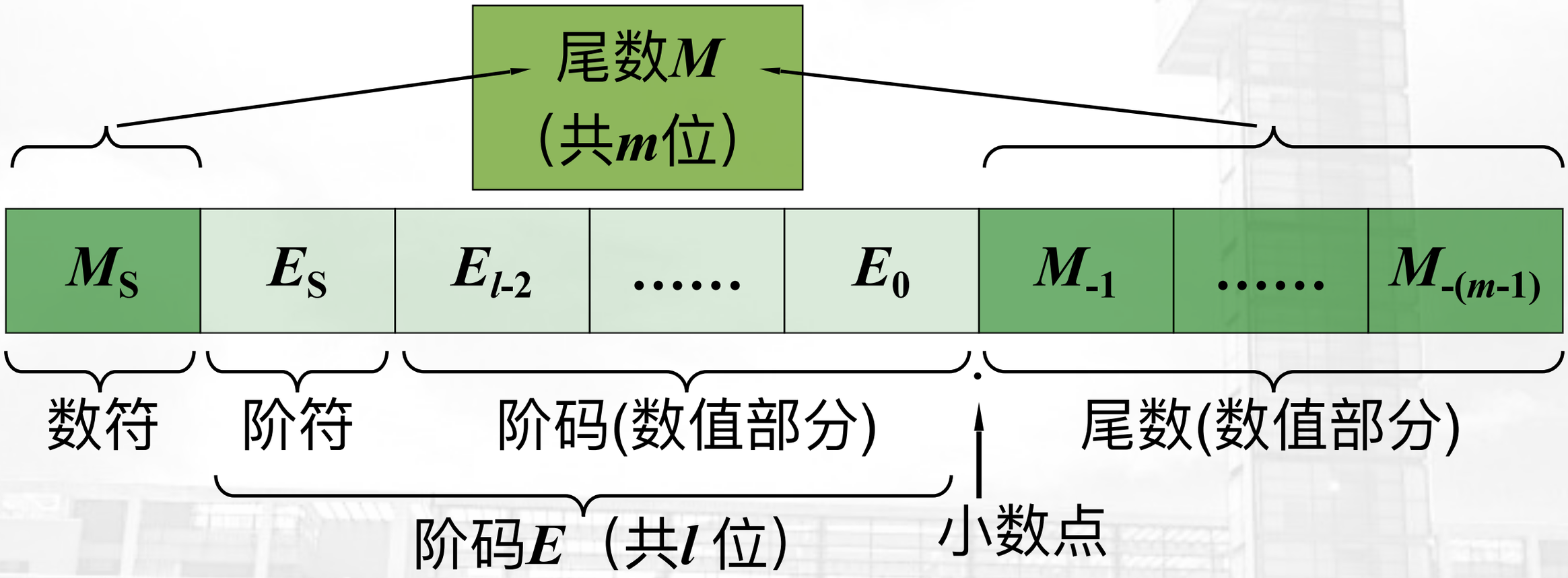
∴数符$S=0$，阶码$E=(128+17)_{10}=(10010001)_2$

因此$(173507)_2$的该32位浮点数格式为：

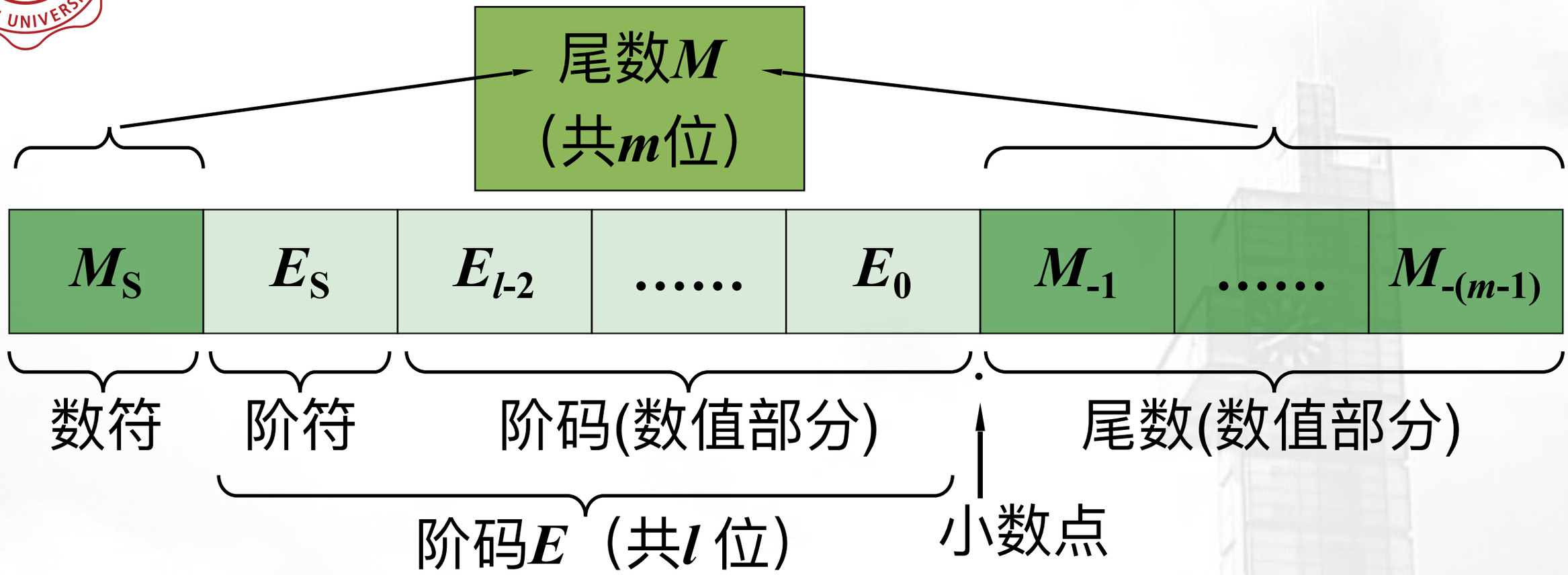
31	30 … 23	22 … 0
0	10010001	010 1001 0111 0000 1100 0000
1位	8位	23位

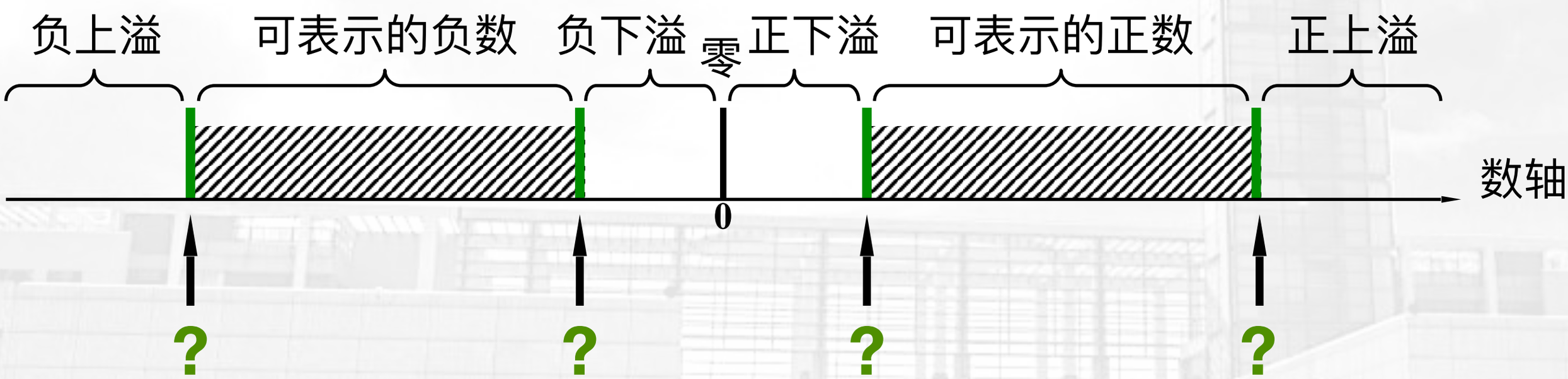
浮点表示——关于表示范围

以2为底，阶码l位（含一位阶符），用移码表示；尾数m位（含一位数符），用规格化/非规格化的补码表示，则浮点数能表示的：最大正数、最小正数、最大负数、最小负数?



浮点表示——关于表示范围


尾数M
(共m位)
M_S
E_S
E_{l-2}
……
E_0
M_{-1}
……
$M_{-(m-1)}$
数符
阶符
阶码(数值部分)
尾数(数值部分)
阶码E (共l 位)
小数点


负上溢
可表示的负数
负下溢
零
正下溢
可表示的正数
正上溢
数轴
0
?
?
?
?

浮点表示——关于表示范围

非规格化版本： 分别考察M和E的最大、最小值，最大正、最大负值并组合他们

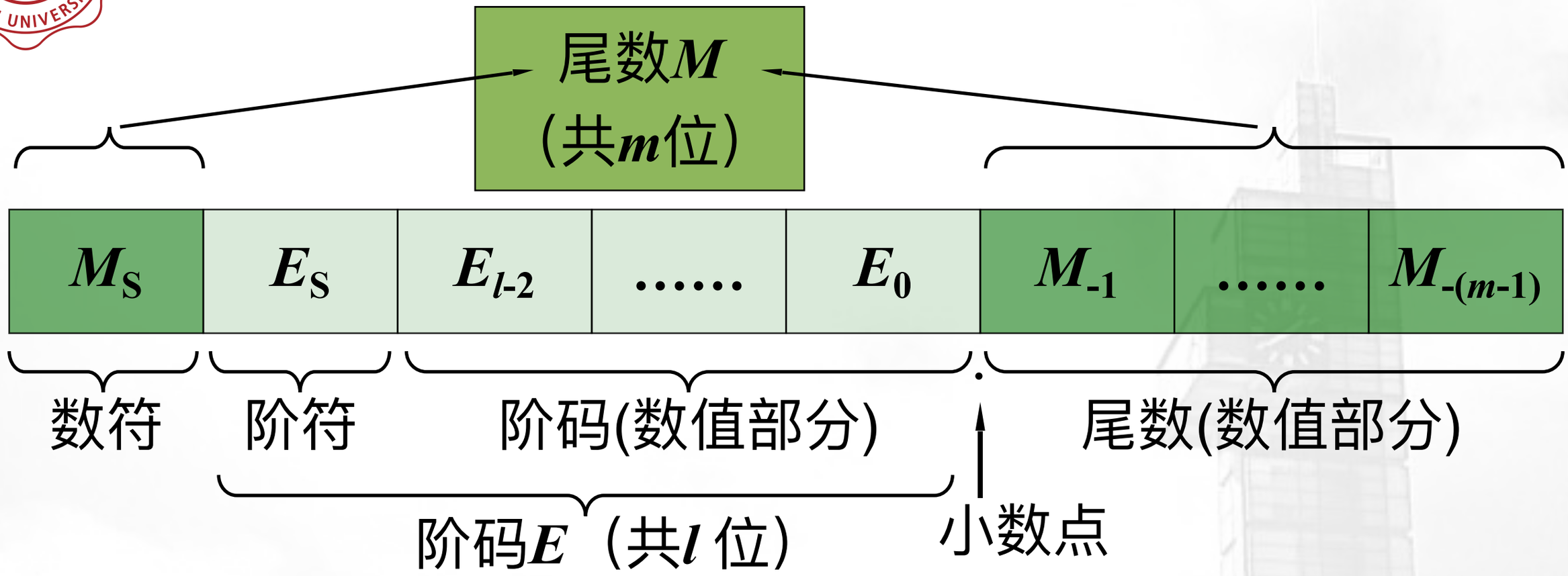
	M（m位补）	E（I位移）	M真值	E真值
最大正值				
最小正值				
最大负值				
最小负值				

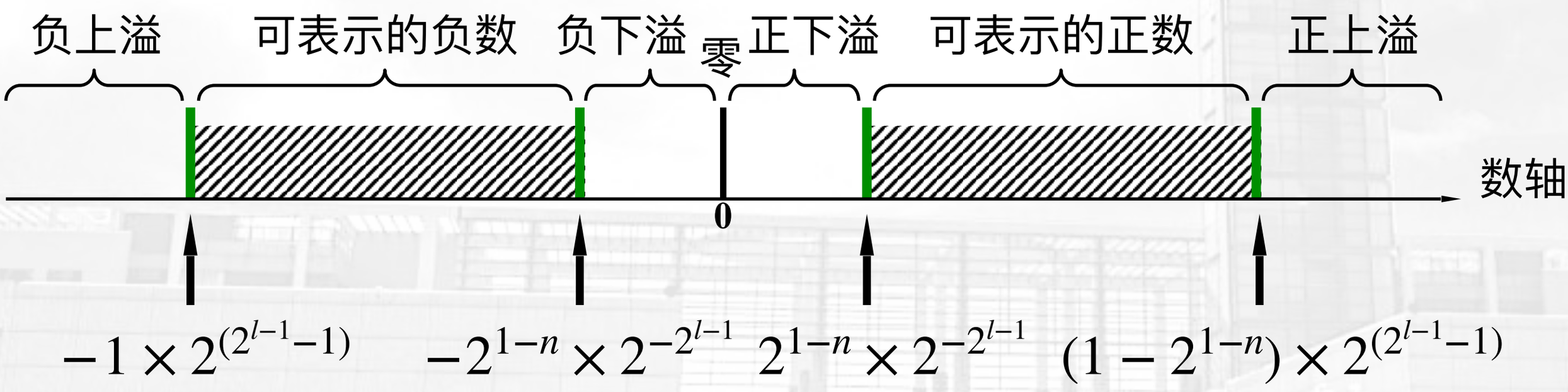
浮点表示——关于表示范围

非规格化版本： 分别考察M和E的最大、最小值，最大正、最大负值并组合他们

	M（n位补）	E（l位移）	M真值	E真值
最大正值	0.111...1111	1111...1111	$1-2^{1-n}$	$2^{l-1}-1$
最小正值	0.000...0001	0000...0000	2^{1-n}	-2^{l-1}
最大负值	1.111...1111	0000...0000	-2^{1-n}	-2^{l-1}
最小负值	1.000...0000	1111...1111	-1	$2^{l-1}-1$

浮点表示——关于表示范围


尾数M
(共m位)
M_S
E_S
E_{l-2}
……
E_0
M_{-1}
……
$M_{-(m-1)}$
数符
阶符
阶码(数值部分)
尾数(数值部分)
阶码E (共l 位)
小数点


负上溢
可表示的负数
负下溢
零
正下溢
可表示的正数
正上溢
数轴
0
$-1\times2^{(2^{l-1}-1)}$
$-2^{1-n}\times2^{-2^{l-1}}$
$2^{1-n}\times2^{-2^{l-1}}$
$(1-2^{1-n})\times2^{(2^{l-1}-1)}$

浮点表示——关于表示范围

规格化版本： 分别考察M和E的最大、最小值，最大正、最大负值并组合他们

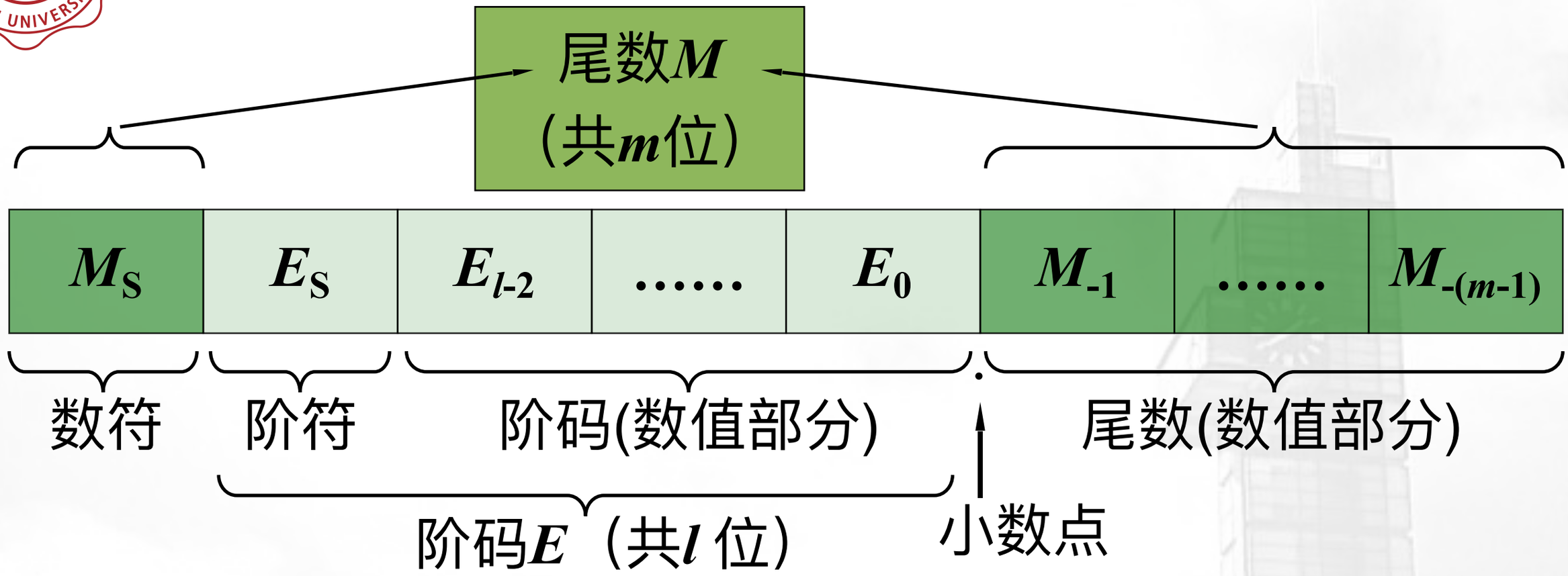
	M（n位补）	E（I位移）	M真值	E真值
最大正值				
最小正值				
最大负值				
最小负值				

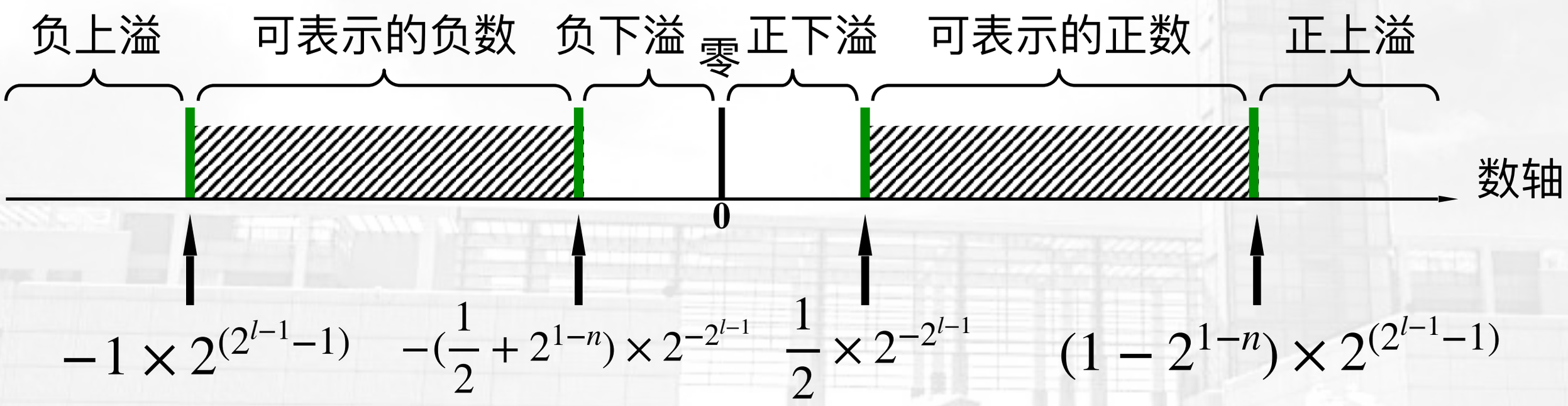
浮点表示——关于表示范围

规格化版本： 分别考察M和E的最大、最小值，最大正、最大负值并组合他们

	M（n位补）	E（l位移）	M真值	E真值
最大正值	0.111...1111	1111...1111	$1-2^{1-n}$	$2^{l-1}-1$
最小正值	0.100...0000	0000...0000	$\frac{1}{2}$	-2^{l-1}
最大负值	1.011...1111	0000...0000	$-(\frac{1}{2}+2^{1-n})$	-2^{l-1}
最小负值	1.000...0000	1111...1111	-1	$2^{l-1}-1$

浮点表示——关于表示范围


尾数M
(共m位)
M_S
E_S
E_{l-2}
……
E_0
M_{-1}
……
$M_{-(m-1)}$
数符
阶符
阶码(数值部分)
尾数(数值部分)
阶码E (共l 位)
小数点


负上溢
可表示的负数
负下溢
零
正下溢
可表示的正数
正上溢
数轴
0
$-1\times2^{(2^{l-1}-1)}$
$-(\frac{1}{2}+2^{1-n})\times2^{-2^{l-1}}$
$\frac{1}{2}\times2^{-2^{l-1}}$
$(1-2^{1-n})\times2^{(2^{l-1}-1)}$

浮点表示—规格化

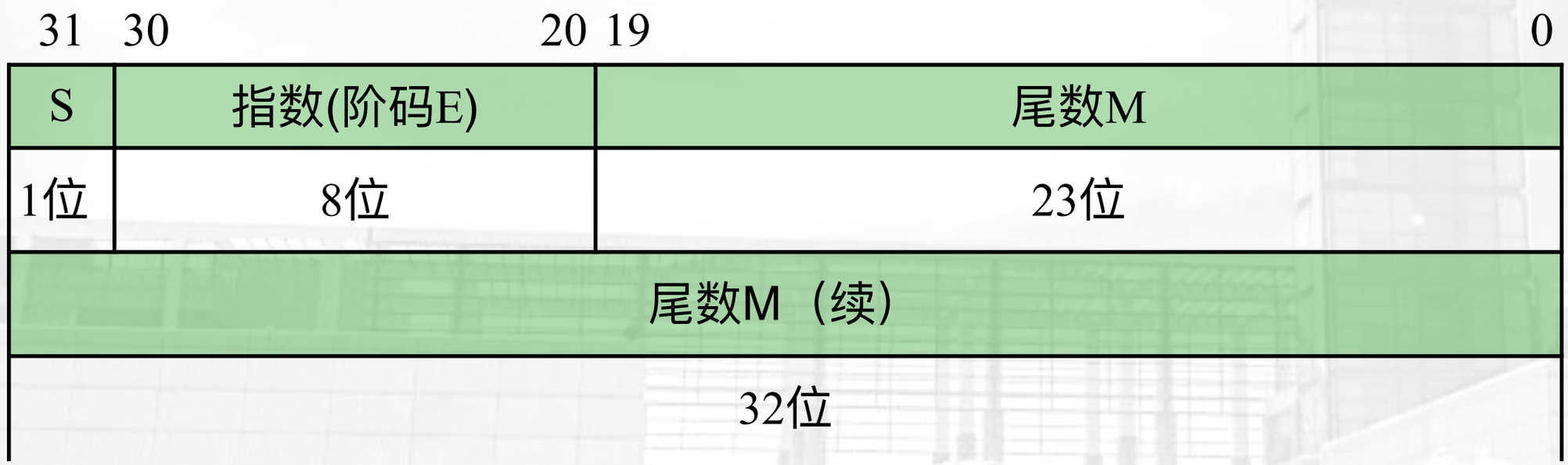
- **机器零**：当浮点数的尾数为0，阶码为最小值全0(移码表示)时，浮点数代码为00…0(全零)。
  - 保证浮点零表示形式的惟一性。
  - 快速判断0
  - 当浮点数的尾数用原码表示时，机器零有“＋0” 和“－0”之分。

浮点表示—两种格式

■ 单精度：**32**位



■ 双精度：**64**位



浮点表示—溢出

- 浮点数的溢出：数的大小超出了浮点数表示范围。原因：指数部分太大，以至于无法用有限的指数字段表示出来。
  - 下溢：阶码小于机器能表示的最小阶码。

    一般当作机器零处理，机器继续运行。
  - 上溢：阶码大于机器能表示的最大阶码。

    机器必须转入溢出故障的中断处理程序进行相应的处理。
- 如何降低上溢或下溢发生的可能性?

  采用双精度格式。范围、精度

  正实数范围：2.78×10-309～9.00×10+307

IEEE-754浮点格式

IEEE-754浮点格式

1. 浮点数的表示标准：IEEE-754，1985年由电子与电气工程师协会(IEEE)制定。
2. 目的：
   - 便于程序从一类处理器移植到另一类处理器上。
   - 研制更为复杂的数值运算程序。
3. 规定：
   - 尾数用原码表示，小数点前隐含一个“1”。
   - 基值隐含为2。
   - 阶码用移码表示，移码的偏移值有专门约定：n位移码的偏移值为 2n-1－1。
   - 指数(阶码)的最大值、最小值作为特殊标记预留，用来标记某些异常事件或机器零。
   - 单精度、双精度；单精度扩展、双精度扩展。

IEEE-754浮点格式

- IEEE-754浮点数表示格式，由IEEE制
  - Never pronounce I-E-E-E，It should be I triple E
- 是国际标准，ISO/ICE/IEEE FDIS 60559:2010
  - 保证普适性，通用性
- IEEE-754是一个非常有“特色”的规格化浮点格式！

IEEE-754浮点格式

- **IEEE-754是一个非常有“特色”的规格化浮点格式！**
  - **尾数用原码表示，但小数点前隐含一个1**
  - **阶码用移码表示，但移码使用非常规的$+2^n-1$格式**
    - **在常规的移码计算之后，额外-1**
- **M、E的一些边界值均有特殊意义，如+/-Inf, NaN, 0等**
- **单精度(float)、双精度（double）、单精度扩展、双精度扩展**

IEEE-754浮点格式

- **IEEE-754** 浮点数表示方法：
  - **000…00** 代表 “＋0”
  - **100…00** 代表 “－0”
  - 其它实数：$(-1)^S\times(1+0.M)\times2^{E\text{-偏移值}}$
- 如果小数部分的尾数各位从左到右依次用 m_1、m_2、m_3、… 来表示，则整个浮点数的值为：

  $(-1)^S\times(1+(m_1\times2^{-1})+(m_2\times2^{-2})+(m_3\times2^{-3})+(m_4\times2^{-4})+\dots)\times2^{E\text{-偏移值}}$
- 为何如此规定浮点数格式：
  - 判断正、负、零；
  - 比较浮点数绝对值的大小。

IEEE-754浮点格式

IEEE 754 格式参数

参数	格式			
	单精度	单精度扩展	双精度	双精度扩展
字宽	32	≥43	64	≥79
尾数有效位数p（包含隐含位）	24	≥32	53	≥64
指数位数	8	≥11	11	≥15
指数偏移值	+127	未指定	+1023	未指定
最大指数E_{max}	+127	≥+1023	+1023	≥+16383
最小指数E_{min}	−126	≤−1022	−1022	≤−16382

IEEE-754浮点格式

IEEE 754单精度浮点数格式

单精度格式位模式	
$0<e<255$	$(-1)^s\times2^{e-127}\times1.f$ (规格化数)
$e=0$；$f\neq0$（f 中至少有一位不为零）	$(-1)^s\times2^{-126}\times0.f$ (非规格化数)
$e=0$；$f=0$（f中所有位均为零）	$(-1)^s\times0.0$ （有符号的零）
$s=0$；$e=255$；$f=0$（f中所有位均为零）	＋INF（正无穷大）
$s=1$；$e=255$；$f=0$（f中所有位均为零）	－INF（负无穷大）
$s=u$；$e=255$；$f\neq0$ （f中至少有一位不为零）	NaN（非数）

31	30 … 23	22 … 0
S	指数(阶码E)	尾数M
1位	8位	23位

IEEE-754浮点格式

IEEE 754单精度浮点数格式

通用名称	浮点数的 IEEE754 编码（二进制）	十进制真值
+0	0 00000000 000...00	+0.0
−0	1 00000000 000...00	−0.0
1	0 01111111 000...00	1.0
2	0 10000000 000...00	2.0
最大规格化数	0 11111110 111...11	$3.40282347\times10^{+38}$
最小正规格化数	0 00000001 000...00	1.17549435×10^{-38}
最大非规格化数	0 00000000 111...11	1.17549421×10^{-38}
最小正非规格化数	0 00000000 000...01	1.40129846×10^{-45}
+∞	0 11111111 000...00	正无穷
−∞	1 11111111 000...00	负无穷
非数	0 11111111 100...00	NaN（不惟一）

IEEE-754浮点格式

IEEE 754 双精度格式位模式表示的值

双精度格式位模式	IEEE浮点数的值
$0<e<2047$	$(-1)^{s}\times 2^{e-1023}\times 1.f$ （规格化数）
$e=0$；$f\neq 0$ （f中至少有一位不为零）	$(-1)^{s}\times 2^{-1022}\times 0.f$ （非规格化数）
$e=0$；$f=0$ （f中所有位均为零）	$(-1)^{s}\times 0.0$ （有符号的零）
$s=0$；$e=2047$；$f=0$ （f中所有位均为零）	＋INF （正无穷大）
$s=1$；$e=2047$；$f=0$ （f中所有位均为零）	－INF （负无穷大）
$s=u$；$e=2047$；$f\neq 0$ （f中至少有一位不为零）	NaN （非数）

IEEE-754浮点格式

IEEE 754双精度格式二进制位与其对应的浮点数真值举例

通用名称	浮点数的IEEE 754编码（十六进制）	十进制真值
+0	00000000 00000000	+0.0
−0	80000000 00000000	−0.0
1	3ff00000 00000000	1.0
2	40000000 00000000	2.0
最大规格化数	7fefffff ffffffff	$1.7976931348623157\times10^{+308}$
最小正规格化数	00100000 00000000	$2.2250738585072014\times10^{-308}$
最大非规格化数	000fffff ffffffff	$2.2250738585072009\times10^{-308}$
最小正非规格化数	00000000 00000001	$4.9406564584124654\times10^{-324}$
+∞	7ff00000 00000000	正无穷
−∞	fff00000 00000000	负无穷
非数	7ff80000 00000000	NaN（不惟一）



2.1.2 数据的定点与浮点表示　3. IEEE 754 标准

【例】若浮点数 x 的 **IEEE 754** 单精度存储格式为 $(\mathbf{42150000})_{16}$，求浮点数 x 的十进制真值。

2.1.2 数据的定点与浮点表示　3. IEEE 754 标准

【例】若浮点数 x 的 **IEEE 754** 单精度存储格式为$(42150000)_{16}$，求浮点数 x 的十进制真值。

【解】将16进制数展开后，可得二进制数格式如下：

0100 0010 0001 0101 0000 0000 0000 0000

S　阶码(8位)　尾数(23位)

指数＝阶码e $-(127)_{10}$

$=(10000100)_2-(01111111)_2=(00000101)_2=(5)_{10}$

包括隐含位1的尾数＝$1.f$

$=(1.001\ 0101\ 0000\ 0000\ 0000\ 0000)_2=(1.001\ 0101)_2$

因此 $x=(-1)^s\times2^{e-127}\times1.f$

$=+2^5\times(1.0010101)_2=(100101.01)_2=(37.25)_{10}$

【例】将数$(22.78125)_{10}$转换成 IEEE 754 单精度浮点数的二进制存储格式。

2.1.2 数据的定点与浮点表示　3. IEEE 754 标准

【例】将数$(22.78125)_{10}$转换成 IEEE 754 单精度浮点数的二进制存储格式。

【解】

$(22.78125)_{10}=(10110.11001)_2$

$=(1.011011001)_2\times 2^4$　　指数　尾数

符号位 S＝0

阶码 e ＝指数＋$(127)_{10}=(4+127)_{10}=(10000011)_2$

尾数 $f=(011011001)_2$

∴ 32位单精度浮点数的二进制存储格式为：

$(0100\ 0001\ 1011\ 0110\ 0100\ 0000\ 0000\ 0000)_2$

$=(41B64000)_{16}$